

Contemporary C++: *Learning Modern C++ in a Modern Way*



مدرس: سعید امراللهی بیوکی

Saeed Amrollahi Boyouki

www.Linkedin.com/in/saeed-amrollahi-boyouki

Agenda 9/30

Some fundamental abstractions: Constants, Pointers, and References

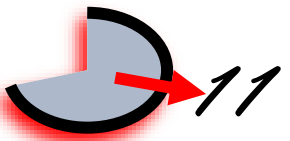
Part 2

- Constants
- Pointers
- Null pointer literal: `nullptr`
- References
- References and further readings
- Q & A

120 min (incl. Q & A)



Null pointers in C and C++



- Pointers and null pointers

```
char c = 'a';  
char* p = &c; // '*' means 'pointer to' p points to c, '&' is address operator
```

- C++03 → Null pointer constant
- ✓ An integer constant expression with the value 0, is called a null pointer constant.

- ✓ A null pointer constant is an integral constant expression rvalue of integer type that evaluates to zero.

```
// macros  
#define NULL 0  
#define NULL ((void *)0)  
char* p = NULL;
```

- Zero (0) is an int.

```
char* p = 0; // points to nowhere  
            // no memory allocation yet
```

- No object is allocated with the address 0. 0 acts as a pointer literal, indicating that a pointer doesn't refer to an object.

Nullptr: a literal for null pointers

- Problems with NULL and 0:

↓ *Distinguishing between null and zero.*

- There is no way to write a call to `f(char*)` with a null pointer value without writing an explicit cast (i.e., `f((char*)0)`) or using a named variable.

↪ *Naming null*

- The pointer literal is the keyword *nullptr*.
- C++11 → Null pointer constants

```
void f(int) { /* ... */ }
void f(char*) { /* ... */ }
f(0); // f(int) called
f(NULL); // f(int) called
```

```
// macros
#define NULL 0
#define NULL ((void *)0)
char* p = NULL;
```

```
int x = nullptr; // error
char* pc = 0; // still good
char* pc2 = nullptr; // best
bool b = (pc == pc2); // OK: b = true
f(nullptr); // called f(char*)
class Point { /* ... */ } *pp = nullptr;
```

Reference Types

- The notation T& means *reference to T*.
- A reference is an alternative name for an object.
- Like constants, references must be initialized.

Type Specifier& Reference = Referent;

```
void f()
{
    int i = 0;
    int& ir; // error: reference should be initialized
    int& iref = i; // OK
    int& iref2 = 1; // error: should be initialized with l-value of an object
}
```

- The main use of references is for specifying arguments and return types for functions.

```
const int& ir = 5; //ok
```

```
void swap(int& x, int& y)
{
    int tmp = x; // decl. statement
    x = y;
    y = tmp;
}
```

```
int a = 0, b = 1;
swap(a, b);
```

Pointer vs. Reference

- A Reference must always refer to some object. There is no *Null Reference*.

```
int& ir = 0; // error: null reference?  
int* ip = 0; // ok: null pointer
```

- A Reference should be *initialized*. Pointer doesn't have such restrictions.

```
bool& ir; // error: reference should be initialized?  
int* ip; // ok: uninitialized pointer: valid but risky
```

```
void UseDouble(const double& rd)  
{  
    // ...  
}
```

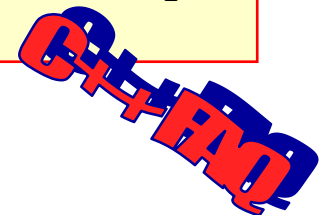
```
void UseDouble(const double* pd)  
{  
    if (pd)  
        // ...  
}
```

- You can't separate reference from referent. You can't reseat reference.

```
string s1("Hello");  
string s2("Good bye");  
string& rs = s1; // rs refers to s1  
string* ps = &s1; // ps points to s1  
rs = s2; // rs still refers to s1, but s1's value is now "Good Bye"  
ps = &s2; // ps not points to s2; s1 is unchanged
```

Q When should I use references, and when should I use pointers?

A Use references when you can, and pointers when you have to.



R references and further readings

[KR78] Brian W. Kernighan & Dennis M. Ritchie. The C Programming Language, Prentice-Hall, Englewood Cliffs, NJ, 1978.

[Str00] Bjarne Stroustrup. The C++ Programming Language (Special Edition). Addison Wesley. Reading Mass. USA. February 2000.

[Str08] Bjarne Stroustrup. Programming: Principles and Practice in C++, 1st edition, Addison Wesley, December 2008.

[Str13] Bjarne Stroustrup. The C++ Programming Language. 4th edition. Addison Wesley. Reading Mass. USA. May 2013.

[Str22] Bjarne Stroustrup. A Tour of C++, 3rd edition, Addison Wesley, September 2022.

Thanks for your patience ...

A man who asks a question is a fool for minute,
The man who does not ask, is a fool for a life.
- Confucius

Learning to ask the right (often hard) questions is an essential part of learning to think as a programmer.

- Bjarne Stroustrup *programming Principles and Practice Using C++, page 4.*

There is no stupid question, but there is stupid answer.
- Howard Hinnant

